Northern University of Bussiness and Technology Khulna

NUBTK $(^w^)$

Sourav Mondal, Rahul Kumar Ghosh, Ayon Adikary Arko

adapted from KACTL and MIT NULL

2025-10-24

1 Contest 1

2 Mathematics 1

3 Data Structures 2

4 Number Theory 2

5 Combinatorial 4

6 Graphs 6

Contest (1)

template.cpp
13 lines

```cpp
#include <bits/stdc++.h>
using namespace std;

#ifdef LOCAL
#include "debug.h"
#else
#define dbg(...)
#endif

int32_t main() {
  cin.tie(0)->sync_with_stdio(0);
  cin.exceptions(cin.failbit);
}
```

.bashrc
5 lines

```bash
# write every function in single line
alias clr="printf '\33c'"
co() { g++ -std=c++23 -O2 -Wall -Wextra
 -Wshadow -Wconversion -o $1 $1.cpp; }
run() { co $1 && ./$1; }
```

hash.sh
3 lines

```bash
# Hash file ignoring whitespace and comments. Verifies that
# code was correctly typed. Usage: 'sh hash.sh < A.cpp'
cpp -dD -P -fpreprocessed|tr -d '[:space:]'|md5sum|cut -c-6
```

stress.sh
20 lines

```bash
#!/usr/bin/env bash

for ((testNum=0;testNum<$4;testNum++))
do
    "./$3".out > input
    "./$2".out < input > outSlow
    "./$1".out < input > outWrong
    if !(cmp -s "outWrong" "outSlow")
    then
        echo "Error found!"
        echo "Input:"
        cat input
        echo "Wrong Output:"
        cat outWrong
        echo "Slow Output:"
        cat outSlow
        exit
    fi
done
echo Passed $4 tests
```

troubleshoot.txt
75 lines

```
General:
* Write down most of your thoughts, even if you're not sure
whether they're useful.
* Give your variables (and files) meaningful names.
* Stay organized and don't leave papers all over the place!
* You should know what your code is doing ...

Pre-submit:
* Write a few simple test cases if sample is not enough.
* Are time limits close? If so, generate max cases.
* Is the memory usage fine?
* Could anything overflow?
* Remove debug output.
* Make sure to submit the right file.

Wrong answer:
* Print your solution! Print debug output as well.
* Read the full problem statement again.
* Have you understood the problem correctly?
* Are you sure your algorithm works?
* Try writing a slow (but correct) solution.
* Can your algorithm handle the whole range of input?
* Did you consider corner cases (ex. n=1)?
* Is your output format correct? (including whitespace)
* Are you clearing all data structures between test cases?
* Any uninitialized variables?
* Any undefined behavior (array out of bounds)?
* Any overflows or NaNs (or shifting ll by >=64 bits)?
* Confusing N and M, i and j, etc.?
* Confusing ++i and i++?
* Return vs continue vs break?
* Are you sure the STL functions you use work as you think?
* Add some assertions, maybe resubmit.
* Create some test cases to run your algorithm on.
* Go through the algorithm for a simple case.
* Go through this list again.
* Explain your algorithm to a teammate.
* Ask the teammate to look at your code.
* Go for a small walk, e.g. to the toilet.
* Rewrite your solution from the start or let a teammate do it.

Geometry:
* Work with ints if possible.
* Correctly account for numbers close to (but not) zero.
  ↪Related:
for functions like acos make sure absolute val of input is not
(slightly) greater than one.
* Correctly deal with vertices that are collinear, concyclic,
coplanar (in 3D), etc.
* Subtracting a point from every other (but not itself)?

Runtime error:
* Have you tested all corner cases locally?
* Any uninitialized variables?
* Are you reading or writing outside the range of any vector?
* Any assertions that might fail?
* Any possible division by 0? (mod 0 for example)
* Any possible infinite recursion?
* Invalidated pointers or iterators?
* Are you using too much memory?
* Debug with resubmits (e.g. remapped signals, see Various).

Time limit exceeded:
* Do you have any possible infinite loops?
* What's your complexity? Large TL does not mean that something
simple (like NlogN) isn't intended.
* Are you copying a lot of unnecessary data? (References)
* Avoid vector, map. (use arrays/unordered_map)
* How big is the input and output? (consider FastIO)
* What do your teammates think about your algorithm?
* Calling count() on multiset?

Memory limit exceeded:
* What is the max amount of memory your algorithm should need?
* Are you clearing all data structures between test cases?
* If using pointers try BumpAllocator.
```

Mathematics (2)

2.1 Trigonometry

$$\sin(v+w) = \sin v \cos w + \cos v \sin w$$
$$\cos(v+w) = \cos v \cos w - \sin v \sin w$$
$$\tan(v+w) = \frac{\tan v + \tan w}{1 - \tan v \tan w}$$
$$\sin v + \sin w = 2 \sin \frac{v+w}{2} \cos \frac{v-w}{2}$$
$$\cos v + \cos w = 2 \cos \frac{v+w}{2} \cos \frac{v-w}{2}$$

$$a \cos x + b \sin x = r \cos(x - \phi)$$
$$a \sin x + b \cos x = r \sin(x + \phi)$$

where $r = \sqrt{a^2 + b^2}, \phi = \operatorname{atan2}(b, a)$.

2.2 Geometry

2.2.1 Triangles

Side lengths: a, b, c

Semiperimeter: $s = \dfrac{a+b+c}{2}$

Area: $A = \sqrt{s(s-a)(s-b)(s-c)}$

Circumradius: $R = \dfrac{abc}{4A}$

Inradius: $r = \dfrac{A}{p}$

Length of median (divides triangle into two equal-area triangles): $m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$

Length of bisector (divides angles in two):

$$s_a = \sqrt{bc\left[1 - \left(\frac{a}{b+c}\right)^2\right]}$$

Law of sines: $\dfrac{\sin\alpha}{a} = \dfrac{\sin\beta}{b} = \dfrac{\sin\gamma}{c} = \dfrac{1}{2R}$

Law of cosines: $a^2 = b^2 + c^2 - 2bc\cos\alpha$

Law of tangents: $\dfrac{a+b}{a-b} = \dfrac{\tan\frac{\alpha+\beta}{2}}{\tan\frac{\alpha-\beta}{2}}$

2.3 Derivatives/Integrals

$$\frac{d}{dx}\arcsin x = \frac{1}{\sqrt{1-x^2}} \qquad \frac{d}{dx}\arccos x = -\frac{1}{\sqrt{1-x^2}}$$

$$\frac{d}{dx}\tan x = 1+\tan^2 x \qquad \frac{d}{dx}\arctan x = \frac{1}{1+x^2}$$

$$\int \tan ax = -\frac{\ln|\cos ax|}{a} \qquad \int x\sin ax = \frac{\sin ax - ax\cos ax}{a^2}$$

$$\int e^{-x^2} = \frac{\sqrt{\pi}}{2}\mathrm{erf}(x) \qquad \int xe^{ax}dx = \frac{e^{ax}}{a^2}(ax-1)$$

Integration by parts:

$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

2.4 Sums/Series

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \ldots, (-1 < x \le 1)$$

$$\sqrt{1+x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \ldots, (-1 \le x \le 1)$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \ldots, (-\infty < x < \infty)$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \ldots, (-\infty < x < \infty)$$

Data Structures (3)

3.1 STL

MapComparator.h

Description: example of function object (functor) for map or set
Usage: `set<int,cmp> s; map<int,int,cmp> m;`

5bfa6c, 1 lines

```
struct cmp{bool operator()(int l,int r)const{return l>r;}};
```

OrderStatisticTree.h

Description: A set (not multiset!) with support for finding the n'th element, and finding the index of an element. Change `null_type` to get a map.
Time: $\mathcal{O}(\log N)$

<ext/pb_ds/assoc_container.hpp>, <ext/pb_ds/assoc_container.hpp> 59e4dd, 18 lines

```
using namespace __gnu_pbds;
template<typename T> using Tree = tree<T, null_type, less<T>,
  ↪rb_tree_tag, tree_order_statistics_node_update>;
#define ook order_of_key
#define fbo find_by_order

using namespace __gnu_pbds;
template<typename T, typename C>
using IndexedSet = tree<T, null_type, less<T, rb_tree_tag,
  ↪tree_order_statistics_node_update>;

void treeExample() {
  Tree<int> t, t2; t.insert(8);
  auto it = t.insert(10).f; assert(it == t.lb(9));
  assert(t.ook(10) == 1 && t.ook(11) == 2 && *t.fbo(0) == 8);
  t.join(t2); // assuming T < T2 or T > T2, merge t2 into t
}
```

3.2 1D Range Queries

RMQ.h

Description: 1D range minimum query. If TL is an issue, use arrays instead of vectors and store values instead of indices.
Memory: $\mathcal{O}(N \log N)$
Time: $\mathcal{O}(1)$

a3f881, 19 lines

```
tcT> struct RMQ { // floor(log_2(x))
  int level(int x) { return 31-__builtin_clz(x); }
  V<T> v; V<vi> jmp;
  int cmb(int a, int b) {
    return v[a]==v[b]?min(a,b):(v[a]<v[b]?a:b); }
  void init(const V<T>& _v) {
    v = _v; jmp = {vi(sz(v))};
    iota(all(jmp[0]),0);
    for (int j = 1; 1<<j <= sz(v); ++j) {
      jmp.pb(vi(sz(v)-(1<<j)+1));
      F0R(i,sz(jmp[j])) jmp[j][i] = cmb(jmp[j-1][i],
        jmp[j-1][i+(1<<(j-1))]);
    }
  }
  int index(int l, int r) {
    assert(l <= r); int d = level(r-l+1);
    return cmb(jmp[d][l],jmp[d][r-(1<<d)+1]); }
  T query(int l, int r) { return v[index(l,r)]; }
};
```

FenwickTree.h

Description: Computes partial sums a[0] + a[1] + ... + a[pos - 1], and updates single elements a[i], taking the difference between the old and new value. 0-Indexed.
Time: Both operations are $\mathcal{O}(\log N)$.

122df3, 23 lines

```
template<class T> struct FenwickTree {
  int N; vector<T> bit;
  FenwickTree(int _N) { N = _N; bit.resize(N); }
  void add(int p, T x) {
    for (++p; p <= N; p += p&-p) bit[p-1] += x;
  }
  T sum(int l, int r) { return sum(r+1)-sum(l); }
  T sum(int r) {
    T res = 0;
    for(; r; r -= r&-r) res += bit[r-1];
    return res;
  }
  int lower_bound(T sum) {
    if (sum <= 0) return -1;
    int pos = 0;
    for (int pw = 1<<25; pw; pw >>= 1) {
      int npos = pos+pw;
      if (npos <= N && bit[npos-1] < sum)
        pos = npos, sum -= bit[pos-1];
    }
    return pos;
  }
};
```

SegmentTree.h

Description: 1D point update and range query where `cmb` is any associative operation. `seg[1]==query(0,N-1)`.
Time: $\mathcal{O}(\log N)$

749f3e, 29 lines

```
template<class T> struct SegTree {
  const T ID{};
  T cmb(T a, T b) { return a + b; }
  int n; vector<T> seg;
  SegTree(int _n) {
    for (n = 1; n < _n; ) n *= 2;
    seg.assign(2*n, ID);
  }
  void pull(int p) { seg[p] = cmb(seg[2*p], seg[2*p+1]); }
  void update(int p, T val) { // set val at position p
    seg[p += n] = val;
    for (p /= 2; p; p /= 2) pull(p);
  }
  T query(int l, int r) { // zero-indexed, inclusive
    T ra = ID, rb = ID;
    for (l += n, r += n + 1; l < r; l /= 2, r /= 2) {
      if (l & 1) ra = cmb(ra, seg[l++]);
      if (r & 1) rb = cmb(seg[--r], rb);
    }
    return cmb(ra, rb);
  }
  // int first_at_least(int lo, int val, int ind, int l, int r)
    ↪ { // if seg stores max across range
  //   if (r < lo || val > seg[ind]) return -1;
  //   if (l == r) return l;
  //   int m = (l+r)/2;
  //   int res = first_at_least(lo,val,2*ind,l,m); if (res !=
    ↪-1) return res;
  //   return first_at_least(lo,val,2*ind+1,m+1,r);
  // }
};
```

3.3 2D Range Queries

PrefixSum2D.h

Description: calculates rectangle sums in constant time

65b070, 17 lines

```
template <typename T> struct PrefixSum2D {
  vector<vector<T>> sum;
  PrefixSum2D(const vector<vector<T>> &v) {
    int n = v.size(), m = v[0].size();
    sum.assign(n + 1, vector<T>(m + 1, T{}));
    for (int i = 0; i < n; ++i) {
      for (int j = 0; j < m; ++j) {
        sum[i+1][j+1] = v[i][j] - sum[i][j]
                      + sum[i+1][j] + sum[i][j+1];
      }
    }
  }
  T query(int x1, int y1, int x2, int y2) {
    return sum[x2][y2] + sum[x1-1][y1-1]
        - sum[x1-1][y2] - sum[x2][y1-1];
  }
};
```

Number Theory (4)

4.1 Modular Arithmetic

ModIntShort.h

Description: Modular arithmetic. Assumes MOD is prime.
Usage: `Z a = MOD+5; inv(a); // 400000003`

6f3a0f, 34 lines

```
struct Z {
  int v;
  static const int MOD = 998244353;
  explicit operator int() const { return v; }
  Z() : v(0) {}
  Z(int64_t _v) : v(int(_v % MOD)) { v += (v < 0) * MOD; }
  Z &operator+=(Z o) {
    if ((v += o.v) >= MOD) v -= MOD;
    return *this;
  }
  Z &operator-=(Z o) {
    if ((v -= o.v) < 0) v += MOD;
    return *this;
```

```
  }
  Z &operator*=(Z o) {
    v = int((int64_t)v * o.v % MOD);
    return *this;
  }
  Z &operator/=(Z o) {
    return (*this) *= inv(o);
  }
  friend Z pow(Z a, int64_t p) {
    assert(p >= 0);
    return p == 0 ? 1 : pow(a * a, p / 2) * (p & 1 ? a : 1);
  }
  friend Z inv(Z a) {
    assert(a.v != 0);
    return pow(a, MOD - 2);
  }
  friend Z operator+(Z a, Z b) { return a += b; }
  friend Z operator-(Z a, Z b) { return a -= b; }
  friend Z operator*(Z a, Z b) { return a *= b; }
  friend Z operator/(Z a, Z b) { return a /= b; }
};
```

ModFact.h

Description: Combinations modulo a prime MOD. Assumes $2 \le N \le MOD$.
Usage: `F.init(10); F.C(6, 4); // 15`
Time: $\mathcal{O}(N)$

"ModInt.h" 364271, 13 lines

```
struct {
  vmi invs, fac, ifac;
  void init(int N) { // idempotent
    invs.rsz(N), fac.rsz(N), ifac.rsz(N);
    invs[1] = fac[0] = ifac[0] = 1;
    FOR(i,2,N) invs[i] = mi(-(ll)MOD/i*(int)invs[MOD%i]);
    FOR(i,1,N) fac[i] = fac[i-1]*i, ifac[i] = ifac[i-1]*invs[i
      ↪];
  }
  mi C(int a, int b) {
    if (a < b || b < 0) return 0;
    return fac[a]*ifac[b]*ifac[a-b];
  }
} F;
```

ModMulLL.h

Description: Multiply two 64-bit integers mod another if 128-bit is not available. modMul is equivalent to `(ul)(__int128(a)*b%mod)`. Works for $0 \le a, b < mod < 2^{63}$. NOTE : Only works if system supports 80-bit floating point.

d22eee, 12 lines

```
using ul = uint64_t;

ul modMul(ul a, ul b, const ul mod) {
  return __int128(a) * __int128(b) % mod;
}

ul modPow(ul a, ul b, const ul mod) {
  if (b == 0) return 1;
  ul res = modPow(a, b / 2, mod);
  res = modMul(res, res, mod);
  return b & 1 ? modMul(res, a, mod) : res;
}
```

FastMod.h

Description: Barrett reduction computes $a\%b$ about 4 times faster than usual where $b > 1$ is constant but not known at compile time. Division by b is replaced by multiplication by m and shifting right 64 bits.

aa19c9, 7 lines

```
using ul = uint64_t; using L = __uint128_t;
struct FastMod {
  ul b, m; FastMod(ul b) : b(b), m(-1ULL / b) {}
  ul reduce(ul a) {
    ul q = (ul)((__uint128_t(m) * a) >> 64), r = a - q * b;
    return r - (r >= b) * b; }
};
```

ModSqrt.h

Description: Tonelli-Shanks algorithm for square roots mod a prime. -1 if doesn't exist.
Usage: `sqrt(mi((ll)1e10)); // 100000`
Time: $\mathcal{O}\left(\log^2(MOD)\right)$

"ModInt.h" bcfa63, 14 lines

```
using T = int;
T sqrt(mi a) {
  mi p = pow(a,(MOD-1)/2);
  if (p.v != 1) return p.v == 0 ? 0 : -1;
  T s = MOD-1; int r = 0; while (s%2 == 0) s /= 2, ++r;
  mi n = 2; while (pow(n,(MOD-1)/2).v == 1) n = T(n)+1;
  // n non-square, ord(g)=2^r, ord(b)=2^m, ord(g)=2^r, m<r
  for (mi x = pow(a,(s+1)/2), b = pow(a,s), g = pow(n,s);;) {
    if (b.v == 1) return min(x.v,MOD-x.v); // x^2=ab
    int m = 0; for (mi t = b; t.v != 1; t *= t) ++m;
    rep(r-m-1) g *= g; // ord(g)=2^{m+1}
    x *= g, g *= g, b *= g, r = m; // ord(g)=2^m, ord(b)<2^m
  }
}
```

ModSum.h

Description: Counts # of lattice points (x, y) in the triangle $1 \le x, 1 \le y, ax + by \le s \pmod{2^{64}}$ and related quantities.
Time: $\mathcal{O}(\log ab)$

23cbf6, 20 lines

```
using ul = uint64_t;
ul sum2(ul n) { return n/2*((n-1)|1); } // sum(0..n-1)
// \return |{(x,y) | 1 <= x, 1 <= y, a*x+b*y <= S}|
//        = sum_{i=1}^{qs} (S-a*i)/b
ul triSum(ul a, ul b, ul s) { assert(a > 0 && b > 0);
  ul qs = s/a, rs = s%a; // ans = sum_{i=0}^{qs-1}(i*a+rs)/b
  ul ad = a/b*sum2(qs)+rs/b*qs; a %= b, rs %= b;
  return ad+(a?triSum(b,a,a*qs+rs):0); // reduce if a >= b
} // then swap x and y axes and recurse

// \return sum_{x=0}^{n-1} (a*x+b)/m
//        = |{(x,y) | 0 < m*y <= a*x+b < a*n+b}|
// assuming a*n+b does not overflow
ul divSum(ul n, ul a, ul b, ul m) { assert(m > 0);
  ul extra = b/m*n; b %= m;
  return extra+(a?triSum(m,a,a*n+b):0); }
// \return sum_{x=0}^{n-1} (a*x+b)%m
ul modSum(ul n, ll a, ll b, ul m) { assert(m > 0);
  a = (a%m+m)%m, b = (b%m+m)%m;
  return a*sum2(n)+b*n-m*divSum(n,a,b,m); }
```

4.2 Primality

4.2.1 Primes

$p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power p^a, except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^\times$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

4.2.2 Divisors

$\sum_{d|n} d = O(n \log \log n)$.

The number of divisors of n is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200 000 for $n < 1e19$.

Dirichlet Convolution: Given a function $f(x)$, let

$(f * g)(x) = \sum_{d|x} g(d) f(x/d)$.

If the partial sums $s_{f*g}(n), s_g(n)$ can be computed in $O(1)$ and $s_f(1 \ldots n^{2/3})$ can be computed in $O\left(n^{2/3}\right)$ then all $s_f\left(\frac{n}{d}\right)$ can as well. Use

$$s_{f*g}(n) = \sum_{d=1}^{n} g(d) s_f(n/d).$$

If $f(x) = \mu(x)$ then $g(x) = 1, (f * g)(x) = (x == 1)$, and $s_f(n) = 1 - \sum_{i=2}^{n} s_f(n/i)$.

If $f(x) = \phi(x)$ then $g(x) = 1, (f * g)(x) = x$, and $s_f(n) = \frac{n(n+1)}{2} - \sum_{i=2}^{n} s_f(n/i)$.

Sieve.h

Description: Tests primality up to SZ. Runs faster if only odd indices are stored.
Time: $\mathcal{O}(SZ \log \log SZ)$ or $\mathcal{O}(SZ)$

41c6ed, 20 lines

```
template<int SZ> struct Sieve {
  bitset<SZ> is_prime; vi primes;
  Sieve() {
    is_prime.set(); is_prime[0] = is_prime[1] = 0;
    for (int i = 4; i < SZ; i += 2) is_prime[i] = 0;
    for (int i = 3; i*i < SZ; i += 2) if (is_prime[i])
      for (int j = i*i; j < SZ; j += i*2) is_prime[j] = 0;
    F0R(i,SZ) if (is_prime[i]) primes.pb(i);
  }
  // int sp[SZ]{}; // smallest prime that divides
  // Sieve() { // above is faster
  //  FOR(i,2,SZ) {
  //    if (sp[i] == 0) sp[i] = i, primes.pb(i);
  //    for (int p: primes) {
  //      if (p > sp[i] || i*p >= SZ) break;
  //      sp[i*p] = p;
  //    }
  //  }
  // }
};
```

MultiplicativePrefixSums.h

Description: $\sum_{i=1}^{N} f(i)$ where $f(i) = \prod \mathtt{val}[e]$ for each p^e in the factorization of i. Must satisfy $\mathtt{val}[1] = 1$. Generalizes to any multiplicative function with $f(p) = p^{\text{fixed power}}$.
Time: $\mathcal{O}\left(\sqrt{N}\right)$

"Sieve.h" 3151ea, 12 lines

```
vmi val;
mi get_prefix(ll N, int p = 0) {
  mi ans = N;
  for (; S.primes.at(p) <= N / S.primes.at(p); ++p) {
    ll new_N = N / S.primes.at(p) / S.primes.at(p);
    for (int idx = 2; new_N; ++idx, new_N /= S.primes.at(p)) {
      ans += (val.at(idx) - val.at(idx - 1))
```

```
          * get_prefix(new_N, p + 1);
    }
  }
  return ans;
}
```

PrimeCnt.h

Description: Counts number of primes up to N. Can also count sum of primes.

Time: $\mathcal{O}\left(N^{3/4}/\log N\right)$, 60ms for $N = 10^{11}$, 2.5s for $N = 10^{13}$

c04e96, 20 lines

```
ll count_primes(ll N) { // count_primes(1e13) == 346065536839
  if (N <= 1) return 0;
  int sq = (int)sqrt(N);
  vl big_ans((sq+1)/2), small_ans(sq+1);
  FOR(i,1,sq+1) small_ans[i] = (i-1)/2;
  F0R(i,sz(big_ans)) big_ans[i] = (N/(2*i+1)-1)/2;
  vb skip(sq+1); int prime_cnt = 0;
  for (int p = 3; p <= sq; p += 2) if (!skip[p]) { // primes
    for (int j = p; j <= sq; j += 2*p) skip[j] = 1;
    F0R(j,min((ll)sz(big_ans),(N/p/p+1)/2)) {
      ll prod = (ll)(2*j+1)*p;
      big_ans[j] -= (prod > sq ? small_ans[(double)N/prod]
            : big_ans[prod/2])-prime_cnt;
    }
    for (int j = sq, q = sq/p; q >= p; --q) for (;j >= q*p;--j)
      small_ans[j] -= small_ans[q]-prime_cnt;
    ++prime_cnt;
  }
  return big_ans[0]+1;
}
```

MillerRabin.h

Description: Deterministic primality test, works up to 2^{64}. For larger numbers, extend A randomly.

"ModMulLL.h" ef770a, 12 lines

```
bool prime(ul n) {
  if (n < 2 || n % 6 % 4 != 1) return n - 2 < 2;
  ul s = __builtin_ctzll(n-1), d = n >> s;
  ul A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022};
  for (auto &a : A) {
    ul p = modPow(a, d, n), i = s;
    while (p != 1 && p != n - 1 && a % n && i--)
      p = modMul(p, p, n);
    if (p != n - 1 && i != s) return false;
  }
  return true;
}
```

FactorBasic.h

Description: Factors integers.

Time: $\mathcal{O}\left(\sqrt{N}\right)$

316cf3, 38 lines

```
inline namespace factorBasic {
template<class T> vector<pair<T, int>> factor(T x) {
  vector<pair<T, int>> pf;
  for (T i = 2; i*i <= x; ++i) if (x%i == 0) {
    int t = 0;
    while (x%i == 0) x /= i, t ++;
    pf.push_back({i, t});
  }
  if (x > 1) pf.push_back({x,1});
  return pf;
}
/* Note:
 * number of operations needed s.t.
 *          phi(phi(...phi(n)...))=1
 * is O(log n).
 * Euler's theorem: a^{\phi(p)}\equiv 1 (mod p), gcd(a,p)=1
 */
ll phi(ll x) {
  each(a,factor(x)) x -= x/a.f;
  return x;
}
template<class T> void tour(vector<pair<T,int>>& v,
  vector<T>& V, int ind, T cur) {
    if (ind == sz(v)) V.pb(cur);
    else {
      T mul = 1;
      F0R(i,v[ind].s+1) {
        tour(v,V,ind+1,cur*mul);
        mul *= v[ind].f;
      }
    }
  }
template<class T> vector<T> getDivi(T x) {
  auto v = factor(x);
  vector<T> V; tour(v,V,0,(T)1); sort(all(V));
  return V;
}
}
```

4.3 Euclidean Algorithm

FracInterval.h

Description: Given fractions $a < b$ with non-negative numerators and denominators, finds fraction f with lowest denominator such that $a < f < b$. Should work with all numbers less than 2^{62}.

1860f3, 6 lines

```
pl bet(pl a, pl b) {
  ll num = a.f/a.s; a.f -= num*a.s, b.f -= num*b.s;
  if (b.f > b.s) return {1+num,1};
  auto x = bet({b.s,b.f},{a.s,a.f});
  return {x.s+num*x.f,x.f};
}
```

Euclid.h

Description: Generalized Euclidean algorithm. `euclid` and `invGeneral` work for $A, B < 2^{62}$.

Time: $\mathcal{O}(\log AB)$

c7e528, 9 lines

```
// ceil(a/b)
// ll cdiv(ll a, ll b) { return a/b+((a^b)>0&&a%b); }
pl euclid(ll A, ll B) { // For A,B>=0, finds (x,y) s.t.
  // Ax+By=gcd(A,B), |Ax|,|By|<=AB/gcd(A,B)
  if (!B) return {1,0};
  pl p = euclid(B,A%B); return {p.s,p.f-A/B*p.s}; }
ll invGeneral(ll A, ll B) { // find x in [0,B) such that Ax=1
  ↪mod B
  pl p = euclid(A,B); assert(p.f*A+p.s*B == 1);
  return p.f+(p.f<0)*B; } // must have gcd(A,B)=1
```

CRT.h

Description: Chinese Remainder Theorem. $a.f \pmod{a.s}, b.f \pmod{b.s} \implies ? \pmod{\mathrm{lcm}(a.s, b.s)}$. Should work for $ab < 2^{62}$.

"Euclid.h" 23df64, 10 lines

```
pl CRT(pl a, pl b) { assert(0 <= a.f && a.f < a.s && 0 <= b.f
  ↪&& b.f < b.s);
  if (a.s < b.s) swap(a,b); // will overflow if b.s^2 > 2^{62}
  ll x,y; tie(x,y) = euclid(a.s,b.s);
  ll g = a.s*x+b.s*y, l = a.s/g*b.s;
  if ((b.f-a.f)%g) return {-1,-1}; // no solution
  // ?*a.s+a.f \equiv b.f \pmod{b.s}
  // ?=(b.f-a.f)/g*(a.s/g)^{-1} \pmod{b.s/g}
  x = (b.f-a.f)%b.s*x%b.s/g*a.s+a.f;
  return {x+(x<0)*l,l};
}
```

ModArith.h

Description: Statistics on mod'ed arithmetic series. `minBetween` and `minRemainder` both assume that $0 \le L \le R < B$, $AB < 2^{62}$

f68a6d, 40 lines

```
ll minBetween(ll A, ll B, ll L, ll R) {
  // min x s.t. exists y s.t. L <= A*x-B*y <= R
  A %= B;
  if (L == 0) return 0;
  if (A == 0) return -1;
  ll k = cdiv(L,A); if (A*k <= R) return k;
  ll x = minBetween(B,A,A-R%A,A-L%A); // min x s.t. exists y
  // s.t. -R <= Bx-Ay <= -L
  return x == -1 ? x : cdiv(B*x+L,A); // solve for y
}

// find min((Ax+C)%B) for 0 <= x <= M
// aka find minimum non-negative value of A*x-B*y+C
// where 0 <= x <= M, 0 <= y
ll minRemainder(ll A, ll B, ll C, ll M) {
  assert(A >= 0 && B > 0 && C >= 0 && M >= 0);
  A %= B, C %= B; ckmin(M,B-1);
  if (A == 0) return C;
  if (C >= A) { // make sure C<A
    ll ad = cdiv(B-C,A);
    M -= ad; if (M < 0) return C;
    C += ad*A-B;
  }
  ll q = B/A, new_B = B%A; // new_B < A
  if (new_B == 0) return C; // B-q*A

  // now minimize A*x-new_B*y+C
  // where 0 <= x,y and x+q*y <= M, 0 <= C < new_B < A
  // q*y -> C-new_B*y
  if (C/new_B > M/q) return C-M/q*new_B;
  M -= C/new_B*q; C %= new_B; // now C < new_B

  // given y, we can compute x = ceil[((B-q*A)*y-C)/A]
  // so x+q*y = ceil((B*y-C)/A) <= M
  ll max_Y = (M*A+C)/B; // must have y <= max_Y
  ll max_X = cdiv(new_B*max_Y-C,A); // must have x <= max_X
  if (max_X*A-new_B*max_Y+C >= new_B) --max_X;
  // now we can remove upper bound on y
  return minRemainder(A,new_B,C,max_X);
}
```

4.4 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2), \quad b = k \cdot (2mn), \quad c = k \cdot (m^2 + n^2),$$

with $m > n > 0$, $k > 0$, $m \perp n$, and either m or n even.

4.5 Lifting the Exponent

For $n > 0$, p prime, and ints x, y s.t. $p \nmid x, y$ and $p | x - y$:

- $p \neq 2$ or $p = 2, 4 | x - y \implies v_p(x^n - y^n) = v_p(x - y) + v_p(n)$.
- $p = 2, 2 | n \implies v_2(x^n - y^n) = v_2((x^2)^{n/2} - (y^2)^{n/2})$.

Combinatorial (5)

5.1 Permutations

5.1.1 Cycles

Let $g_S(n)$ be the number of n-permutations whose cycle lengths all belong to the set S. Then

$$\sum_{n=0}^{\infty} g_S(n)\frac{x^n}{n!} = \exp\left(\sum_{n\in S}\frac{x^n}{n}\right)$$

5.1.2 Burnside's lemma

Given a group G of symmetries and a set X, the number of elements of X *up to symmetry* equals

$$\frac{1}{|G|}\sum_{g\in G}|X^g|,$$

where X^g are the elements fixed by g ($g.x = x$).

If $f(n)$ counts "configurations" (of some sort) of length n, we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n}\sum_{k=0}^{n-1} f(\gcd(n,k)) = \frac{1}{n}\sum_{k|n} f(k)\phi(n/k).$$

5.2 Partitions and subsets

5.2.1 Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1,\ p(n) = \sum_{k\in\mathbb{Z}\setminus\{0\}} (-1)^{k+1} p(n - k(3k-1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	~2e5	~2e8

5.2.2 Lucas' Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_k p^k + ... + n_1 p + n_0$ and $m = m_k p^k + ... + m_1 p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^{k}\binom{n_i}{m_i} \pmod p$.

5.3 General purpose numbers

5.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t-1}$ (FFT-able). $B[0,\ldots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \ldots]$

Sums of powers:

$$\sum_{i=1}^{n} i^m = \frac{1}{m+1}\sum_{k=0}^{m}\binom{m+1}{k}B_k(n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\sum_{i=m}^{\infty} f(i) = \int_m^{\infty} f(x)dx - \sum_{k=1}^{\infty}\frac{B_k}{k!}f^{(k-1)}(m)$$

$$\approx \int_m^{\infty} f(x)dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m))$$

5.3.2 Stirling numbers of the first kind

Number of permutations on n items with k cycles.

$$c(n,k) = c(n-1,k-1) + (n-1)c(n-1,k),\ c(0,0) = 1$$
$$\textstyle\sum_{k=0}^{n} c(n,k)x^k = x(x+1)\ldots(x+n-1)$$

$c(8,k) = 8, 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$
$c(n,2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \ldots$

5.3.3 Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. k j:s s.t. $\pi(j) > \pi(j+1)$, $k+1$ j:s s.t. $\pi(j) \geq j$, k j:s s.t. $\pi(j) > j$.

$$E(n,k) = (n-k)E(n-1,k-1) + (k+1)E(n-1,k)$$

$$E(n,0) = E(n,n-1) = 1$$

$$E(n,k) = \sum_{j=0}^{k}(-1)^j\binom{n+1}{j}(k+1-j)^n$$

5.3.4 Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$S(n,k) = S(n-1,k-1) + kS(n-1,k)$$

$$S(n,1) = S(n,n) = 1$$

$$S(n,k) = \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n$$

5.3.5 Bell numbers

Total number of partitions of n distinct elements. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \ldots$. For p prime,

$$B(p^m + n) \equiv mB(n) + B(n+1) \pmod p$$

5.3.6 Labeled unrooted trees

\# on n vertices: n^{n-2}
\# on k existing trees of size n_i: $n_1 n_2 \cdots n_k n^{k-2}$
\# with degrees d_i: $(n-2)!/((d_1-1)!\cdots(d_n-1)!)$

5.3.7 Catalan numbers

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1,\ C_{n+1} = \frac{2(2n+1)}{n+2}C_n,\ C_{n+1} = \sum C_i C_{n-i}$$

$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \ldots$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with n pairs of parenthesis, correctly nested.
- binary trees with with $n+1$ leaves (0 or 2 children).
- ordered trees with $n+1$ vertices.
- ways a convex polygon with $n+2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

5.4 Young Tableaux

Let a **Young diagram** have shape $\lambda = (\lambda_1 \geq \cdots \geq \lambda_k)$, where λ_i equals the number of cells in the i-th (left-justified) row from the top. A **Young tableau** of shape λ is a filling of the $n = \sum\lambda_i$ cells with a permutation of $1\ldots n$ such that each row and column is increasing.

Hook-Length Formula: For the cell in position (i,j), let $h_\lambda(i,j) = |\{(I,J)|i \leq I, j \leq J, (I = i \text{ or } J = j)\}|$. The number of Young tableaux of shape λ is equal to $f^\lambda = \frac{n!}{\prod h_\lambda(i,j)}$.

Schensted's Algorithm: converts a permutation σ of length n into a pair of Young Tableaux $(S(\sigma), T(\sigma))$ of the same shape. When inserting $x = \sigma_i$,

1. Add x to the first row of S by inserting x in place of the largest y with $x < y$. If y doesn't exist, push x to the end of the row, set the value of T at that position to be i, and stop.
2. Add y to the second row using the same rule, keep repeating as necessary.

All pairs $(S(\sigma), T(\sigma))$ of the same shape correspond to a unique σ, so $n! = \sum(f^\lambda)^2$. Also, $S(\sigma^R) = S(\sigma)^T$.

Let $d_k(\sigma), a_k(\sigma)$ be the lengths of the longest subseqs which are a union of k decreasing/ascending subseqs, respectively. Then $a_k(\sigma) = \sum_{i=1}^{k}\lambda_i, d_k(\sigma) = \sum_{i=1}^{k}\lambda_i^*$, where λ_i^* is size of the i-th column.

5.5 Other

DeBruijnSeq.h

Description: Given alphabet $[0,k)$ constructs a cyclic string of length k^n that contains every length n string as substr.

a6961b, 13 lines

```
vi deBruijnSeq(int k, int n) {
  if (k == 1) return {0};
  vi seq, aux(n+1);
  function<void(int,int)> gen = [&](int t, int p) {
```

```
    if (t > n) { // +lyndon word of len p
      if (n%p == 0) FOR(i,1,p+1) seq.pb(aux[i]);
    } else {
      aux[t] = aux[t-p]; gen(t+1,p);
      while (++aux[t] < k) gen(t+1,t);
    }
  };
  gen(1,1); return seq;
}
```

NimProduct.h

Description: Product of nimbers is associative, commutative, and distributive over addition (xor). Forms finite field of size 2^{2^k}. Defined by $ab = \text{mex}(\{a'b + ab' + a'b' : a' < a, b' < b\})$. Application: Given 1D coin turning games G_1, G_2 $G_1 \times G_2$ is the 2D coin turning game defined as follows. If turning coins at $x_1, x_2, \ldots, x_m$ is legal in G_1 and $y_1, y_2, \ldots, y_n$ is legal in G_2, then turning coins at all positions (x_i, y_j) is legal assuming that the coin at (x_m, y_n) goes from heads to tails. Then the grundy function $g(x, y)$ of $G_1 \times G_2$ is $g_1(x) \times g_2(y)$.
Time: 64^2 xors per multiplication, memorize to speed up.

5afe17, 46 lines

```
using ul = uint64_t;
struct Precalc {
  ul tmp[64][64], y[8][8][256];
  unsigned char x[256][256];
  Precalc() { // small nim products, all < 256
    F0R(i,256) F0R(j,256) x[i][j] = mult<8>(i,j);
    F0R(i,8) F0R(j,i+1) F0R(k,256)
      y[i][j][k] = mult<64>(prod2(8*i,8*j),k);
  }
  ul prod2(int i, int j) { // nim prod of 2^i, 2^j
    ul& u = tmp[i][j]; if (u) return u;
    if (!(i&j)) return u = 1ULL<<(i|j);
    int a = (i&j)&-(i&j); // a=2^k, consider 2^{2^k}
    return u=prod2(i^a,j)^prod2((i^a)|(a-1),(j^a)|(i&(a-1)));
    // 2^{2^k}*2^{2^k} = 2^{2^k}+2^{2^k-1}
  } // 2^{2^i}*2^{2^j} = 2^{2^i+2^j} if i<j
  template<int L> ul mult(ul a, ul b) {
    ul c = 0; F0R(i,L) if (a>>i&1)
      F0R(j,L) if (b>>j&1) c ^= prod2(i,j);
    return c;
  }
  // 2^{8*i}*(a>>(8*i)&255) * 2^{8*j}*(b>>(8*j)&255)
  // -> (2^{8*i}*2^{8*j})*((a>>(8*i)&255)*(b>>(8*j)&255))
  ul multFast(ul a, ul b) const { // faster nim product
    ul res = 0; auto f=[](ul c,int d) {return c>>(8*d)&255;};
    F0R(i,8) {
      F0R(j,i) res ^= y[i][j][x[f(a,i)][f(b,j)]
               ^x[f(a,j)][f(b,i)]];
      res ^= y[i][i][x[f(a,i)][f(b,i)]];
    }
    return res;
  }
};
const Precalc P;

struct nb { // nimber
  ul x; nb() { x = 0; }
  nb(ul _x): x(_x) {}
  explicit operator ul() { return x; }
  nb operator+(nb y) { return nb(x^y.x); }
  nb operator*(nb y) { return nb(P.multFast(x,y.x)); }
  friend nb pow(nb b, ul p) {
    nb res = 1; for (;p;p/=2,b=b*b) if (p&1) res = res*b;
    return res; } // b^{2^{2^A}-1}=1 where 2^{2^A} > b
  friend nb inv(nb b) { return pow(b,-2); }
};
```

MatroidIsect.h

Description: Computes a set of maximum size which is independent in both graphic and colorful matroids, aka a spanning forest where no two edges are of the same color. In general, construct the exchange graph and find a shortest path. Can apply similar concept to partition matroid.
Usage: `MatroidIsect<Gmat,Cmat> M(sz(ed),Gmat(ed),Cmat(col))`
Time: $\mathcal{O}\left(GI^{1.5}\right)$ calls to oracles, where G is size of ground set and I is size of independent set.

"DSU.h" d0051c, 51 lines

```
struct Gmat { // graphic matroid
  int V = 0; vpi ed; DSU D;
  Gmat(vpi _ed):ed(_ed) {
    map<int,int> m; each(t,ed) m[t.f] = m[t.s] = 0;
    each(t,m) t.s = V++;
    each(t,ed) t.f = m[t.f], t.s = m[t.s];
  }
  void clear() { D.init(V); }
  void ins(int i) { assert(D.unite(ed[i].f,ed[i].s)); }
  bool indep(int i) { return !D.sameSet(ed[i].f,ed[i].s); }
};
struct Cmat { // colorful matroid
  int C = 0; vi col; V<bool> used;
  Cmat(vi col):col(col) {each(t,col) ckmax(C,t+1); }
  void clear() { used.assign(C,0); }
  void ins(int i) { used[col[i]] = 1; }
  bool indep(int i) { return !used[col[i]]; }
};
template<class M1, class M2> struct MatroidIsect {
  int n; V<bool> iset; M1 m1; M2 m2;
  bool augment() {
    vi pre(n+1,-1); queue<int> q({n});
    while (sz(q)) {
      int x = q.ft; q.pop();
      if (iset[x]) {
        m1.clear(); F0R(i,n) if (iset[i] && i != x) m1.ins(i);
        F0R(i,n) if (!iset[i] && pre[i] == -1 && m1.indep(i))
          pre[i] = x, q.push(i);
      } else {
        auto backE = [&]() { // back edge
          m2.clear();
          F0R(c,2)F0R(i,n)if((x==i||iset[i])&&(pre[i]==-1)==c){
            if (!m2.indep(i))return c?pre[i]=x,q.push(i),i:-1;
            m2.ins(i); }
          return n;
        };
        for (int y; (y = backE()) != -1;) if (y == n) {
          for(; x != n; x = pre[x]) iset[x] = !iset[x];
          return 1; }
      }
    }
    return 0;
  }
  MatroidIsect(int n, M1 m1, M2 m2):n(n), m1(m1), m2(m2) {
    iset.assign(n+1,0); iset[n] = 1;
    m1.clear(); m2.clear(); // greedily add to basis
    R0F(i,n) if (m1.indep(i) && m2.indep(i))
      iset[i] = 1, m1.ins(i), m2.ins(i);
    while (augment());
  }
};
```

Graphs (6)

Erdos-Gallai: $d_1 \geq \cdots \geq d_n$ can be degree sequence of simple graph on n vertices iff their sum is even and $\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k), \forall 1 \leq k \leq n$.

6.1 Basics

DSU.h

Description: Disjoint Set Union with path compression and union by size. Add edges and test connectivity. Use for Kruskal's or Boruvka's minimum spanning tree.
Time: $\mathcal{O}(\alpha(N))$

a876ce, 12 lines

```
struct DSU {
  vector<int> e;
  DSU(int N) { e = vector<int>(N, -1); }
  int get(int x) { return e[x] < 0 ? x : e[x] = get(e[x]); }
  bool sameSet(int a, int b) { return get(a) == get(b); }
  int size(int x) { return -e[get(x)]; }
  bool unite(int x, int y) { // union by size
    x = get(x), y = get(y); if (x == y) return false;
    if (e[x] > e[y]) swap(x, y);
    e[x] += e[y]; e[y] = x; return true;
  }
};
```

NegativeCycle.h

Description: use Bellman-Ford (make sure no underflow)

688ec8, 11 lines

```
vi negCyc(int N, V<pair<pi,int>> ed) {
  vl d(N); vi p(N); int x = -1;
  rep(N) {
    x = -1; each(t,ed) if (ckmin(d[t.f.s],d[t.f.f]+t.s))
      p[t.f.s] = t.f.f, x = t.f.s;
    if (x == -1) return {};
  }
  rep(N) x = p[x]; // enter cycle
  vi cyc{x}; while (p[cyc.bk] != x) cyc.pb(p[cyc.bk]);
  reverse(all(cyc)); return cyc;
}
```

Dijkstra.h

Description: shortest path
Time: $\mathcal{O}(N \log N + M)$

0a9b8e, 33 lines

```
template<class T, bool directed> struct Dijkstra {
  int SZ;
  vector<T> dist;
  vector<vector<pair<int, T>>> adj;
  Dijkstra(int _SZ) {
    SZ = _SZ;
    adj.resize(SZ);
  }
  void ae(int u, int v, T w) {
    adj[u].push_back({v, w});
    if (!directed) adj[v].push_back({u, w});
  }
  void run(int st) {
    dist.assign(SZ, numeric_limits<T>::max());
    priority_queue<pair<T, int>> PQ;
    auto relax = [&](int v, T d) {
      if (dist[v] <= d) return;
      dist[v] = d;
      PQ.push({-dist[v], v});
    }; relax(st, 0);
    while (!PQ.empty()) {
      auto [d, u] = PQ.top(); PQ.pop(); d = -d;
      if (dist[u] < d) continue;
      for (auto& [v, w]: adj[u]) relax(v, w+d);
    }
  }
  // vector<int> getPath(int v) {
  //   vector<int> path;
  //   for (; v != -1; v = par[v]) path.push_back(v);
  //   ranges::reverse(path);
```

```
  //   return path;
  // }
};
```

6.2 Trees

LCAjump.h

Description: Calculates least common ancestor in tree with verts $0 \ldots N-1$ and root R using binary jumping.
Memory: $\mathcal{O}(N \log N)$
Time: $\mathcal{O}(N \log N)$ build, $\mathcal{O}(\log N)$ query

2edfbe, 47 lines

```
struct LCA {
  int N, LOG;
  vector<int> depth;
  vector<vector<int>> par, adj;
  LCA(int _N) {
    N = _N; LOG = 1;
    while ((1<<LOG) < N) ++LOG;
    adj.resize(N);
    depth.resize(N);
    par.resize(N, vector<int>(LOG));
  }
  void ae(int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
  }
  void build(int R = 0) { par[R][0] = R; dfs(R); }
  void dfs(int u = 0) {
    for (int j = 1; j < LOG; ++j) {
      par[u][j] = par[par[u][j-1]][j-1];
    }
    for (auto& v: adj[u]) {
      if (v == par[u][0]) continue;
      depth[v] = depth[u]+1;
      par[v][0] = u;
      dfs(v);
    }
  }
  int jump(int x, int d) {
    for (int j = 0; j < LOG; ++j) {
      if ((d>>j) & 1) x = par[x][j];
    }
    return x;
  }
  int lca(int u, int v) {
    if (depth[u] > depth[v]) swap(u, v);
    v = jump(v, depth[v]-depth[u]);
    if (u == v) {return u; }
    for (int j = LOG-1; j >= 0; --j) {
      int U = par[u][j], V = par[v][j];
      if (U != V) u = U, v = V;
    }
    return par[u][0];
  }
  int dist(int u, int v) {
    return depth[u] + depth[v] - 2*depth[lca(u, v)];
  }
```

6.2.1 SqrtDecompton

HLD generally suffices. If not, here are some common strategies:

- Rebuild the tree after every $\sqrt{N}$ queries.
- Consider vertices with $>$ or $< \sqrt{N}$ degree separately.
- For subtree updates, note that there are $O(\sqrt{N})$ distinct sizes among child subtrees of any node.

Block Tree: Use a DFS to split edges into contiguous groups of size $\sqrt{N}$ to $2\sqrt{N}$.

Mo's Algorithm for Tree Paths: Maintain an array of vertices where each one appears twice, once when a DFS enters the vertex (`st`) and one when the DFS exists (`en`). For a tree path $u \leftrightarrow v$ such that `st[u]<st[v]`,

- If u is an ancestor of v, query `[st[u],st[v]]`.
- Otherwise, query `[en[u],st[v]]` and consider $LCA(u, v)$ separately.

Solutions with worse complexities can be faster if you optimize the operations that are performed most frequently. Use arrays instead of vectors whenever possible. Iterating over an array in order is faster than iterating through the same array in some other order (ex. one given by a random permutation) or DFSing on a tree of the same size. Also, the difference between $\sqrt{N}$ and the optimal block (or buffer) size can be quite large. Try up to 5x smaller or larger (at least).

6.3 DFS Algorithms

DFSBasic.h

Description: Basic DFS implementation for undirected graphs
Time: $\mathcal{O}(N + M)$

14b944, 21 lines

```
struct DFS {
  int N;
  vector<bool> visited;
  vector<vector<int>> adj;
  DFS(int _N) {
    N = _N;
    adj.resize(N);
    visited.resize(N);
  }
  void ae(int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
  }
  void dfs(int u) {
    visited[u] = true;
    for (auto& v: adj[u]) {
      if (visited[v]) continue;
      dfs(v);
    }
  }
};
```

EulerPath.h

Description: Eulerian path starting at `src` if it exists, visits all edges exactly once. Works for both directed and undirected. Returns vector of {vertex,label of edge to vertex}. Second element of first pair is always -1.
Time: $\mathcal{O}(N + M)$

9c222d, 23 lines

```
template<bool directed> struct Euler {
  int N; V<vpi> adj; V<vpi::iterator> its; vb used;
  void init(int _N) { N = _N; adj.rsz(N); }
  void ae(int a, int b) {
    int M = sz(used); used.pb(0);
    adj[a].eb(b,M); if (!directed) adj[b].eb(a,M); }
  vpi solve(int src = 0) {
    its.rsz(N); F0R(i,N) its[i] = begin(adj[i]);
    vpi ans, s{{src,-1}}; // {{vert,prev vert},edge label}
    int lst = -1; // ans generated in reverse order
    while (sz(s)) {
      int x = s.bk.f; auto& it=its[x], en=end(adj[x]);
      while (it != en && used[it->s]) ++it;
      if (it == en) { // no more edges out of vertex
        if (lst != -1 && lst != x) return {};
        // not a path, no tour exists
        ans.pb(s.bk); s.pop_back(); if (sz(s)) lst=s.bk.f;
      } else s.pb(*it), used[it->s] = 1;
    } // must use all edges
    if (sz(ans) != sz(used)+1) return {};
    reverse(all(ans)); return ans;
  }
};
```

SCCT.h

Description: Tarjan's, DFS once to generate strongly connected components in topological order. a, b in same component if both $a \to b$ and $b \to a$ exist. Uses less memory than Kosaraju b/c doesn't store reverse edges.
Time: $\mathcal{O}(N + M)$

a36e0c, 22 lines

```
struct SCC {
  int N, ti = 0; V<vi> adj;
  vi disc, comp, stk, comps;
  void init(int _N){ N = _N, adj.rsz(N);
    disc.rsz(N), comp.rsz(N,-1);}
  void ae(int x, int y) { adj[x].pb(y); }
  int dfs(int x) {
    int low = disc[x] = ++ti; stk.pb(x);
    each(y,adj[x]) if (comp[y] == -1) // comp[y] == -1,
      ckmin(low,disc[y]?:dfs(y));  // disc[y] != 0 -> in stack
    if (low == disc[x]) { // make new SCC
      // pop off stack until you find x
      comps.pb(x); for (int y = -1; y != x;)
        comp[y = stk.bk] = x, stk.pop_back();
    }
    return low;
  }
  void gen() {
    F0R(i,N) if (!disc[i]) dfs(i);
    reverse(all(comps));
  }
};
```

TwoSAT.h

Description: Calculates a valid assignment to boolean variables a, b, c,... to a 2-SAT problem, so that an expression of the type $(a\|\|b)\&\&(!a\|\|c)\&\&(d\|\|!b)\&\&\ldots$ becomes true, or reports that it is unsatisfiable. Negated variables are represented by bit-inversions (~x).
Usage: `TwoSat ts;`

```
ts.either(0, ~3); // Var 0 is true or var 3 is false
ts.setVal(2); // Var 2 is true
ts.atMostOne({0,~1,2}); // <= 1 of vars 0, ~1 and 2 are true
ts.solve(N); // Returns true iff it is solvable
ts.ans[0..N-1] holds the assigned values to the vars
```

"SCC.h" ff0f3d, 31 lines

```
struct TwoSAT {
  int N = 0; vpi edges;
  void init(int _N) { N = _N; }
  int addVar() { return N++; }
  void either(int x, int y) {
    x = max(2*x,-1-2*x), y = max(2*y,-1-2*y);
    edges.eb(x,y); }
  void implies(int x, int y) { either(~x,y); }
  void must(int x) { either(x,x); }
  void atMostOne(const vi& li) {
```

```
    if (sz(li) <= 1) return;
    int cur = ~li[0];
    FOR(i,2,sz(li)) {
      int next = addVar();
      either(cur,~li[i]); either(cur,next);
      either(~li[i],next); cur = ~next;
    }
    either(cur,~li[1]);
  }
  vb solve() {
    SCC S; S.init(2*N);
    each(e,edges) S.ae(e.f^1,e.s), S.ae(e.s^1,e.f);
    S.gen(); reverse(all(S.comps)); // reverse topo order
    for (int i = 0; i < 2*N; i += 2)
      if (S.comp[i] == S.comp[i^1]) return {};
    vi tmp(2*N); each(i,S.comps) if (!tmp[i])
      tmp[i] = 1, tmp[S.comp[i^1]] = -1;
    vb ans(N); F0R(i,N) ans[i] = tmp[S.comp[2*i]] == 1;
    return ans;
  }
};
```

BCC.h

Description: Biconnected components of edges. Removing any vertex in BCC doesn't disconnect it. To get block-cut tree, create a bipartite graph with the original vertices on the left and a vertex for each BCC on the right. Draw edge $u \leftrightarrow v$ if u is contained within the BCC for v. Self-loops are not included in any BCC while BCCS of size 1 represent bridges.
Time: $\mathcal{O}(N+M)$

0625a6, 55 lines

```
struct BCC
{
  V<vpi> adj;
  vpi ed;
  V<vi> edgeSets, vertSets; // edges for each bcc
  int N, ti = 0;
  vi disc, stk;
  void init(int _N)
  {
    N = _N;
    disc.rsz(N), adj.rsz(N);
  }
  void ae(int x, int y)
  {
    adj[x].eb(y, sz(ed)), adj[y].eb(x, sz(ed)), ed.eb(x, y);
  }
  int dfs(int x, int p = -1)
  { // return lowest disc
    int low = disc[x] = ++ti;
    each(e, adj[x]) if (e.s != p)
    {
      if (!disc[e.f])
      {
        stk.pb(e.s); // disc[x] < LOW -> bridge
        int LOW = dfs(e.f, e.s);
        ckmin(low, LOW);
        if (disc[x] <= LOW)
        { // get edges in bcc
          edgeSets.eb();
          vi &tmp = edgeSets.bk; // new bcc
          for (int y = -1; y != e.s;)
            tmp.pb(y = stk.bk), stk.pop_back();
        }
      }
      else if (disc[e.f] < disc[x]) // back-edge
        ckmin(low, disc[e.f]), stk.pb(e.s);
    }
    return low;
  }
  void gen()
  {
    F0R(i, N)
    if (!disc[i]) dfs(i);
    vb in(N);
    each(c, edgeSets)
    {                  // edges contained within each BCC
      vertSets.eb(); // so you can easily create block cut tree
      auto ad = [&](int x)
      {
        if (!in[x]) in[x] = 1, vertSets.bk.pb(x); };
      each(e, c) ad(ed[e].f), ad(ed[e].s);
      each(e, c) in[ed[e].f] = in[ed[e].s] = 0;
    }
  }
};
```

MaximalCliques.h

Description: Used only once. Finds all maximal cliques.
Time: $\mathcal{O}\left(3^{N/3}\right)$

f5cd93, 16 lines

```
using B = bitset<128>; B adj[128];
int N;
// possibly in clique, not in clique, in clique
void cliques(B P = ~B(), B X={}, B R={}) {
  if (!P.any()) {
    if (!X.any()) // do smth with R
    return;
  }
  int q = (P|X)._Find_first();
  // clique must contain q or non-neighbor of q
  B cands = P&~adj[q];
  F0R(i,N) if (cands[i]) {
    R[i] = 1; cliques(P&adj[i],X&adj[i],R);
    R[i] = P[i] = 0; X[i] = 1;
  }
}
```

6.4 Flows

Konig's Theorem: In a bipartite graph, max matching = min vertex cover.

Dilworth's Theorem: For any partially ordered set, the sizes of the max antichain and of the min chain decomposition are equal. Equivalent to Konig's theorem on the bipartite graph (U, V, E) where $U = V = S$ and (u, v) is an edge when $u < v$. Those vertices outside the min vertex cover in both U and V form a max antichain.

Dinic.h

Description: Fast flow. After computing flow, edges $\{u, v\}$ such that $lev[u] \neq -1$, $lev[v] = -1$ are part of min cut. Use reset and rcap for Gomory-Hu.
Time: $\mathcal{O}(N^2M)$ flow, $\mathcal{O}\left(M\sqrt{N}\right)$ bipartite matching

b7b370, 38 lines

```
struct Dinic {
  using F = ll; // flow type
  struct Edge { int to; F flo, cap; };
  int N; V<Edge> eds; V<vi> adj;
  void init(int _N) { N = _N; adj.rsz(N), cur.rsz(N); }
  void ae(int u, int v, F cap, F rcap = 0) { assert(min(cap,
    ↪rcap) >= 0);
    adj[u].pb(sz(eds)); eds.pb({v,0,cap});
    adj[v].pb(sz(eds)); eds.pb({u,0,rcap});
  }
  vi lev; V<vi::iterator> cur;
  bool bfs(int s, int t) { // level = shortest distance from
    ↪source
    lev = vi(N,-1); F0R(i,N) cur[i] = begin(adj[i]);
    queue<int> q({s}); lev[s] = 0;
    while (sz(q)) { int u = q.ft; q.pop();
      each(e,adj[u]) { const Edge& E = eds[e];
        int v = E.to; if (lev[v] < 0 && E.flo < E.cap)
          q.push(v), lev[v] = lev[u]+1;
      }
    }
    return lev[t] >= 0;
  }
  F dfs(int v, int t, F flo) {
    if (v == t) return flo;
    for (; cur[v] != end(adj[v]); cur[v]++) {
      Edge& E = eds[*cur[v]];
      if (lev[E.to]!=lev[v]+1||E.flo==E.cap) continue;
      F df = dfs(E.to,t,min(flo,E.cap-E.flo));
      if (df) { E.flo += df; eds[*cur[v]^1].flo -= df;
        return df; } // saturated >=1 one edge
    }
    return 0;
  }
  F maxFlow(int s, int t) {
    F tot = 0; while (bfs(s,t)) while (F df =
      dfs(s,t,numeric_limits<F>::max())) tot += df;
    return tot;
  }
};
```

GomoryHu.h

Description: Returns edges of Gomory-Hu tree (second element is weight). Max flow between pair of vertices of undirected graph is given by min edge weight along tree path. Uses the fact that for any i, j, k, $\lambda_{ik} \geq \min(\lambda_{ij}, \lambda_{jk})$, where λ_{ij} denotes the flow between i and j.
Time: $N-1$ calls to Dinic

"Dinic.h" 0d712e, 16 lines

```
template<class F> V<pair<pi,F>> gomoryHu(int N,
    const V<pair<pi,F>>& ed) {
  vi par(N); Dinic<F> D; D.init(N);
  vpi ed_locs; each(t,ed)ed_locs.pb(D.ae(t.f.f,t.f.s,t.s,t.s));
  V<pair<pi,F>> ans;
  FOR(i,1,N) {
    each(p,ed_locs) { // reset capacities
      auto& e = D.adj.at(p.f).at(p.s);
      auto& e_rev = D.adj.at(e.to).at(e.rev);
      e.cap = e_rev.cap = (e.cap+e_rev.cap)/2;
    }
    ans.pb({{i,par[i]},D.maxFlow(i,par[i])});
    FOR(j,i+1,N) if (par[j] == par[i] && D.lev[j]) par[j] = i;
  }
  return ans;
}
```

MCMF.h

Description: Minimum-cost maximum flow, assumes no negative cycles. It is possible to choose negative edge costs such that the first run of Dijkstra is slow, but this hasn't been an issue in the past. Edge weights ≥ 0 for every subsequent run. To get flow through original edges, assign ID's during ae.
Time: Ignoring first run of Dijkstra, $\mathcal{O}(FM \log M)$ if caps are integers and F is max flow.

072c1b, 40 lines

```
struct MCMF {
  using F = ll; using C = ll; // flow type, cost type
  struct Edge { int to; F flo, cap; C cost; };
  int N; V<C> p, dist; vi pre; V<Edge> eds; V<vi> adj;
  void init(int _N) { N = _N;
    p.rsz(N), dist.rsz(N), pre.rsz(N), adj.rsz(N); }
  void ae(int u, int v, F cap, C cost) { assert(cap >= 0);
```

```
    adj[u].pb(sz(eds)); eds.pb({v,0,cap,cost});
    adj[v].pb(sz(eds)); eds.pb({u,0,0,-cost});
  } // use asserts, don't try smth dumb
  bool path(int s, int t) { // find lowest cost path to send flow through
    const C inf = numeric_limits<C>::max(); F0R(i,N) dist[i] = inf;
    using T = pair<C,int>; priority_queue<T,vector<T>,greater<T>> todo;
    todo.push({dist[s] = 0,s});
    while (sz(todo)) { // Dijkstra
      T x = todo.top(); todo.pop(); if (x.f > dist[x.s]) continue;
      each(e,adj[x.s]) { const Edge& E = eds[e]; // all weights should be non-negative
        if (E.flo < E.cap && ckmin(dist[E.to],x.f+E.cost+p[x.s]-p[E.to]))
          pre[E.to] = e, todo.push({dist[E.to],E.to});
      }
    } // if costs are doubles, add some EPS so you
    // don't traverse ~0-weight cycle repeatedly
    return dist[t] != inf; // return flow
  }
  pair<F,C> calc(int s, int t) { assert(s != t);
    F0R(_,N) F0R(e,sz(eds)) { const Edge& E = eds[e]; // Bellman-Ford
      if (E.cap) ckmin(p[E.to],p[eds[e^1].to]+E.cost); }
    F totFlow = 0; C totCost = 0;
    while (path(s,t)) { // p -> potentials for Dijkstra
      F0R(i,N) p[i] += dist[i]; // don't matter for unreachable nodes
      F df = numeric_limits<F>::max();
      for (int x = t; x != s; x = eds[pre[x]^1].to) {
        const Edge& E = eds[pre[x]]; ckmin(df,E.cap-E.flo); }
      totFlow += df; totCost += (p[t]-p[s])*df;
      for (int x = t; x != s; x = eds[pre[x]^1].to)
        eds[pre[x]].flo += df, eds[pre[x]^1].flo -= df;
    } // get max flow you can send along path
    return {totFlow,totCost};
  }
};
```

6.5 Matching

Hungarian.h

Description: Given J jobs and W workers (J <= W), computes the minimum cost to assign each prefix of jobs to distinct workers.
@tparam T a type large enough to represent integers on the order of J * max(—C—) @param C a matrix of dimensions JxW such that C[j][w] = cost to assign j-th job to w-th worker (possibly negative)
@return a vector of length J, with the j-th entry equaling the minimum cost to assign the first (j+1) jobs to distinct workers
Time: $\mathcal{O}\left(J^2W\right)$

f382f7, 36 lines

```
template <class T> vector<T> hungarian(const vector<vector<T>>&C) {
  const int J = (int)size(C), W = (int)size(C[0]);
  assert(J <= W);
  vector<int> job(W + 1, -1);
  vector<T> ys(J), yt(W + 1);
  vector<T> answers;
  const T inf = numeric_limits<T>::max();
  for (int j_cur = 0; j_cur < J; ++j_cur) {
    int w_cur = W;
    job[w_cur] = j_cur;
    vector<T> min_to(W + 1, inf);
    vector<int> prv(W + 1, -1);
    vector<bool> in_Z(W + 1);
    while (job[w_cur] != -1) {
      in_Z[w_cur] = true;
      const int j = job[w_cur];
      T delta = inf;
      int w_next;
      for (int w = 0; w < W; ++w) {
        if (!in_Z[w]) {
          if (ckmin(min_to[w], C[j][w] - ys[j] - yt[w]))
            prv[w] = w_cur;
          if (ckmin(delta, min_to[w])) w_next = w;
        }
      }
      for (int w = 0; w <= W; ++w) {
        if (in_Z[w]) ys[job[w]] += delta, yt[w] -= delta;
        else min_to[w] -= delta;
      }
      w_cur = w_next;
    }
    for (int w; w_cur != -1; w_cur = w) job[w_cur] = job[w = prv[w_cur]];
    answers.push_back(-yt[W]);
  }
  return answers;
}
```

GeneralMatchBlossom.h

Description: Variant on Gabow's Impl of Edmond's Blossom Algorithm. General unweighted max matching with 1-based indexing. If `white[v] = 0` after `solve()` returns, `v` is part of every max matching.
Time: $\mathcal{O}(NM)$, faster in practice

fd5cc7, 50 lines

```
struct MaxMatching {
  int N; V<vi> adj;
  V<int> mate, first; vb white; vpi label;
  void init(int _N) { N = _N; adj = V<vi>(N+1);
    mate = first = vi(N+1); label = vpi(N+1); white = vb(N+1);}
  void ae(int u, int v) { adj.at(u).pb(v), adj.at(v).pb(u); }
  int group(int x) { if (white[first[x]]) first[x] = group(first[x]);
    return first[x]; }
  void match(int p, int b) {
    swap(b,mate[p]); if (mate[b] != p) return;
    if (!label[p].s) mate[b] = label[p].f, match(label[p].f,b); // vertex label
    else match(label[p].f,label[p].s), match(label[p].s,label[p].f); // edge label
  }
  bool augment(int st) { assert(st);
    white[st] = 1; first[st] = 0; label[st] = {0,0};
    queue<int> q; q.push(st);
    while (!q.empty()) {
      int a = q.ft; q.pop(); // outer vertex
      each(b,adj[a]) { assert(b);
        if (white[b]) { // two outer vertices, form blossom
          int x = group(a), y = group(b), lca = 0;
          while (x||y) {
            if (y) swap(x,y);
            if (label[x] == pi{a,b}) { lca = x; break; }
            label[x] = {a,b}; x = group(label[mate[x]].first);
          }
          for (int v: {group(a),group(b)}) while (v != lca) {
            assert(!white[v]); // make everything along path white
            q.push(v); white[v] = true; first[v] = lca;
            v = group(label[mate[v]].first);
          }
        } else if (!mate[b]) { // found augmenting path
          mate[b] = a; match(a,b); white = vb(N+1); // reset
          return true;
        } else if (!white[mate[b]]) {
          white[mate[b]] = true; first[mate[b]] = b;
          label[b] = {0,0}; label[mate[b]] = pi{a,0};
          q.push(mate[b]);
        }
      }
    }
    return false;
  }
  int solve() {
    int ans = 0;
    FOR(st,1,N+1) if (!mate[st]) ans += augment(st);
    FOR(st,1,N+1) if (!mate[st] && !white[st]) assert(!augment(st));
    return ans;
  }
};
```

GeneralWeightedMatch.h

Description: General max weight max matching with 1-based indexing. Edge weights must be positive, combo of UnweightedMatch and Hungarian.
Time: $\mathcal{O}\left(N^3\right)$?

120873, 145 lines

```
template<int SZ> struct WeightedMatch {
  struct edge { int u,v,w; }; edge g[SZ*2][SZ*2];
  void ae(int u, int v, int w) { g[u][v].w = g[v][u].w = w; }
  int N,NX,lab[SZ*2],match[SZ*2],slack[SZ*2],st[SZ*2];
  int par[SZ*2],floFrom[SZ*2][SZ],S[SZ*2],aux[SZ*2];
  vi flo[SZ*2]; queue<int> q;
  void init(int _N) { N = _N; // init all edges
    FOR(u,1,N+1) FOR(v,1,N+1) g[u][v] = {u,v,0}; }
  int eDelta(edge e) { // >= 0 at all times
    return lab[e.u]+lab[e.v]-g[e.u][e.v].w*2; }
  void updSlack(int u, int x) { // smallest edge -> blossom x
    if (!slack[x] || eDelta(g[u][x]) < eDelta(g[slack[x]][x]))
      slack[x] = u; }
  void setSlack(int x) {
    slack[x] = 0; FOR(u,1,N+1) if (g[u][x].w > 0
     && st[u] != x && S[st[u]] == 0) updSlack(u,x); }
  void qPush(int x) {
    if (x <= N) q.push(x);
    else each(t,flo[x]) qPush(t); }
  void setSt(int x, int b) {
    st[x] = b; if (x > N) each(t,flo[x]) setSt(t,b); }
  int getPr(int b, int xr) { // get even position of xr
    int pr = find(all(flo[b]),xr)-begin(flo[b]);
    if (pr&1) { reverse(1+all(flo[b])); return sz(flo[b])-pr; }
    return pr; }
  void setMatch(int u, int v) { // rearrange flo[u], matches
    edge e = g[u][v]; match[u] = e.v; if (u <= N) return;
    int xr = floFrom[u][e.u], pr = getPr(u,xr);
    F0R(i,pr) setMatch(flo[u][i],flo[u][i^1]);
    setMatch(xr,v); rotate(begin(flo[u]),pr+all(flo[u])); }
  void augment(int u, int v) { // set matches including u->v
    while (1) { // and previous ones
      int xnv = st[match[u]]; setMatch(u,v);
      if (!xnv) return;
      setMatch(xnv,st[par[xnv]]);
      u = st[par[xnv]], v = xnv;
    }
  }
  int lca(int u, int v) { // same as in unweighted
    static int t = 0; // except maybe return 0
    for (++t;u||v;swap(u,v)) {
      if (!u) continue;
      if (aux[u] == t) return u;
      aux[u] = t; u = st[match[u]];
      if (u) u = st[par[u]];
    }
    return 0;
```

```
  }
  void addBlossom(int u, int anc, int v) {
    int b = N+1; while (b <= NX && st[b]) ++b;
    if (b > NX) ++NX; // new blossom
    lab[b] = S[b] = 0; match[b] = match[anc]; flo[b] = {anc};
    auto blossom = [&](int x) {
      for (int y; x != anc; x = st[par[y]])
        flo[b].pb(x), flo[b].pb(y = st[match[x]]), qPush(y);
    };
    blossom(u); reverse(1+all(flo[b])); blossom(v); setSt(b,b);
    // identify all nodes in current blossom
    FOR(x,1,NX+1) g[b][x].w = g[x][b].w = 0;
    FOR(x,1,N+1) floFrom[b][x] = 0;
    each(xs,flo[b]) { // find tightest constraints
      FOR(x,1,NX+1) if (g[b][x].w == 0 || eDelta(g[xs][x]) <
        eDelta(g[b][x])) g[b][x]=g[xs][x], g[x][b]=g[x][xs];
      FOR(x,1,N+1) if (floFrom[xs][x]) floFrom[b][x] = xs;
    } // floFrom to deconstruct blossom
    setSlack(b); // since didn't qPush everything
  }
  void expandBlossom(int b) {
    each(t,flo[b]) setSt(t,t); // undo setSt(b,b)
    int xr = floFrom[b][g[b][par[b]].u], pr = getPr(b,xr);
    for(int i = 0; i < pr; i += 2) {
      int xs = flo[b][i], xns = flo[b][i+1];
      par[xs] = g[xns][xs].u; S[xs] = 1; // no setSlack(xns)?
      S[xns] = slack[xs] = slack[xns] = 0; qPush(xns);
    }
    S[xr] = 1, par[xr] = par[b];
    FOR(i,pr+1,sz(flo[b])) { // matches don't change
      int xs = flo[b][i]; S[xs] = -1, setSlack(xs); }
    st[b] = 0; // blossom killed
  }
  bool onFoundEdge(edge e) {
    int u = st[e.u], v = st[e.v];
    if (S[v] == -1) { // v unvisited, matched with smth else
      par[v] = e.u, S[v] = 1; slack[v] = 0;
      int nu = st[match[v]]; S[nu] = slack[nu] = 0; qPush(nu);
    } else if (S[v] == 0) {
      int anc = lca(u,v); // if 0 then match found!
      if (!anc) return augment(u,v),augment(v,u),1;
      addBlossom(u,anc,v);
    }
    return 0;
  }
  bool matching() {
    q = queue<int>();
    FOR(x,1,NX+1) {
      S[x] = -1, slack[x] = 0; // all initially unvisited
      if (st[x] == x && !match[x]) par[x] = S[x] = 0, qPush(x);
    }
    if (!sz(q)) return 0;
    while (1) {
      while (sz(q)) { // unweighted matching with tight edges
        int u = q.ft; q.pop(); if (S[st[u]] == 1) continue;
        FOR(v,1,N+1) if (g[u][v].w > 0 && st[u] != st[v]) {
          if (eDelta(g[u][v]) == 0) { // condition is strict
            if (onFoundEdge(g[u][v])) return 1;
          } else updSlack(u,st[v]);
        }
      }
      int d = INT_MAX;
      FOR(b,N+1,NX+1) if (st[b] == b && S[b] == 1)
        ckmin(d,lab[b]/2); // decrease lab[b]
      FOR(x,1,NX+1) if (st[x] == x && slack[x]) {
        if (S[x] == -1) ckmin(d,eDelta(g[slack[x]][x]));
        else if (S[x] == 0) ckmin(d,eDelta(g[slack[x]][x])/2);
      } // edge weights shouldn't go below 0
      FOR(u,1,N+1) {
        if (S[st[u]] == 0) {
          if (lab[u] <= d) return 0; // why?
          lab[u] -= d;
        } else if (S[st[u]] == 1) lab[u] += d;
      } // lab has opposite meaning for verts and blossoms
      FOR(b,N+1,NX+1) if (st[b] == b && S[b] != -1)
        lab[b] += (S[b] == 0 ? 1 : -1)*d*2;
      q = queue<int>();
      FOR(x,1,NX+1) if (st[x]==x && slack[x] // new tight edge
        && st[slack[x]] != x && eDelta(g[slack[x]][x]) == 0)
          if (onFoundEdge(g[slack[x]][x])) return 1;
      FOR(b,N+1,NX+1) if (st[b]==b && S[b]==1 && lab[b]==0)
        expandBlossom(b); // odd dist blossom taken apart
    }
    return 0;
  }
  pair<ll,int> calc() {
    NX = N; st[0] = 0; FOR(i,1,2*N+1) aux[i] = 0;
    FOR(i,1,N+1) match[i] = 0, st[i] = i, flo[i].clear();
    int wMax = 0;
    FOR(u,1,N+1) FOR(v,1,N+1)
      floFrom[u][v] = (u == v ? u : 0), ckmax(wMax,g[u][v].w);
    FOR(u,1,N+1) lab[u] = wMax; // start high and decrease
    int num = 0; ll wei = 0; while (matching()) ++num;
    FOR(u,1,N+1) if (match[u] && match[u] < u)
      wei += g[u][match[u]].w; // edges in matching
    return {wei,num};
  }
};
```

MaxMatchFast.h

Description: Fast bipartite matching.

Time: $\mathcal{O}\left(M\sqrt{N}\right)$

ec6c96, 31 lines

```
vpi maxMatch(int L, int R, const vpi& edges) {
  V<vi> adj = V<vi>(L);
  vi nxt(L,-1), prv(R,-1), lev, ptr;
  F0R(i,sz(edges)) adj.at(edges[i].f).pb(edges[i].s);
  while (true) {
    lev = ptr = vi(L); int max_lev = 0;
    queue<int> q; F0R(i,L) if (nxt[i]==-1) lev[i]=1, q.push(i);
    while (sz(q)) {
      int x = q.ft; q.pop();
      for (int y: adj[x]) {
        int z = prv[y];
        if (z == -1) max_lev = lev[x];
        else if (!lev[z]) lev[z] = lev[x]+1, q.push(z);
      }
      if (max_lev) break;
    }
    if (!max_lev) break;
    F0R(i,L) if (lev[i] > max_lev) lev[i] = 0;
    auto dfs = [&](auto self, int x) -> bool {
      for (;ptr[x] < sz(adj[x]);++ptr[x]) {
        int y = adj[x][ptr[x]], z = prv[y];
        if (z == -1 || (lev[z] == lev[x]+1 && self(self,z)))
          return nxt[x]=y, prv[y]=x, ptr[x]=sz(adj[x]), 1;
      }
      return 0;
    };
    F0R(i,L) if (nxt[i] == -1) dfs(dfs,i);
  }
  vpi ans; F0R(i,L) if (nxt[i] != -1) ans.pb({i,nxt[i]});
  return ans;
}
```

6.6 Advanced

ChordalGraphRecognition.h

Description: Recognizes graph where every induced cycle has length exactly 3 using maximum adjacency search.

6cc97d, 58 lines

```
int N,M;
set<int> adj[MX];
int cnt[MX];
vi ord, rord;

vi find_path(int x, int y, int z) {
  vi pre(N,-1);
  queue<int> q; q.push(x);
  while (sz(q)) {
    int t = q.ft; q.pop();
    if (adj[t].count(y)) {
      pre[y] = t; vi path = {y};
      while (path.bk != x) path.pb(pre[path.bk]);
      path.pb(z);
      return path;
    }
    each(u,adj[t]) if (u != z && !adj[u].count(z) && pre[u] ==
      ↪-1) {
      pre[u] = t;
      q.push(u);
    }
  }
  assert(0);
}

int main() {
  setIO(); re(N,M);
  F0R(i,M) {
    int a,b; re(a,b);
    adj[a].insert(b), adj[b].insert(a);
  }
  rord = vi(N,-1);
  priority_queue<pi> pq;
  F0R(i,N) pq.push({0,i});
  while (sz(pq)) {
    pi p = pq.top(); pq.pop();
    if (rord[p.s] != -1) continue;
    rord[p.s] = sz(ord); ord.pb(p.s);
    each(t,adj[p.s]) pq.push({++cnt[t],t});
  }
  assert(sz(ord) == N);
  each(z,ord) {
    pi big = {-1,-1};
    each(y,adj[z]) if (rord[y] < rord[z])
      ckmax(big,mp(rord[y],y));
    if (big.f == -1) continue;
    int y = big.s;
    each(x,adj[z]) if (rord[x] < rord[y]) if (!adj[y].count(x))
      ↪ {
      ps("NO");
      vi v = find_path(x,y,z);
      ps(sz(v));
      each(t,v) pr(t,' ');
      exit(0);
    }
  }
  ps("YES");
  reverse(all(ord));
  each(z,ord) pr(z,' ');
}
```

DominatorTree.h

Description: Used only a few times. Assuming that all nodes are reachable from *root*, a dominates b iff every path from *root* to b passes through a.
Time: $\mathcal{O}(M \log N)$

4b8836, 41 lines

```
template<int SZ> struct Dominator {
  vi adj[SZ], ans[SZ]; // input edges, edges of dominator tree
  vi radj[SZ], child[SZ], sdomChild[SZ];
  int label[SZ], rlabel[SZ], sdom[SZ], dom[SZ], co = 0;
  int par[SZ], bes[SZ];
  void ae(int a, int b) { adj[a].pb(b); }
  int get(int x) { // DSU with path compression
    // get vertex with smallest sdom on path to root
    if (par[x] != x) {
      int t = get(par[x]); par[x] = par[par[x]];
      if (sdom[t] < sdom[bes[x]]) bes[x] = t;
    }
    return bes[x];
  }
  void dfs(int x) { // create DFS tree
    label[x] = ++co; rlabel[co] = x;
    sdom[co] = par[co] = bes[co] = co;
    each(y,adj[x]) {
      if (!label[y]) {
        dfs(y); child[label[x]].pb(label[y]); }
      radj[label[y]].pb(label[x]);
    }
  }
  void init(int root) {
    dfs(root);
    ROF(i,1,co+1) {
      each(j,radj[i]) ckmin(sdom[i],sdom[get(j)]);
      if (i > 1) sdomChild[sdom[i]].pb(i);
      each(j,sdomChild[i]) {
        int k = get(j);
        if (sdom[j] == sdom[k]) dom[j] = sdom[j];
        else dom[j] = k;
      }
      each(j,child[i]) par[j] = i;
    }
    FOR(i,2,co+1) {
      if (dom[i] != sdom[i]) dom[i] = dom[dom[i]];
      ans[rlabel[dom[i]]].pb(rlabel[i]);
    }
  }
};
```

EdgeColor.h

Description: Used only once. Naive implementation of Misra & Gries edge coloring. By Vizing's Theorem, a simple graph with max degree d can be edge colored with at most $d+1$ colors
Time: $\mathcal{O}(N^2 M)$, faster in practice

cc2b29, 40 lines

```
template<int SZ> struct EdgeColor {
  int N = 0, maxDeg = 0, adj[SZ][SZ], deg[SZ];
  void init(int _N) { N = _N;
    F0R(i,N) { deg[i] = 0; F0R(j,N) adj[i][j] = 0; } }
  void ae(int a, int b, int c) {
    adj[a][b] = adj[b][a] = c; }
  int delEdge(int a, int b) {
    int c = adj[a][b]; adj[a][b] = adj[b][a] = 0;
    return c; }
  V<bool> genCol(int x) {
    V<bool> col(N+1); F0R(i,N) col[adj[x][i]] = 1;
    return col; }
  int freeCol(int u) {
    auto col = genCol(u); int x = 1;
    while (col[x]) ++x; return x; }
  void invert(int x, int d, int c) {
    F0R(i,N) if (adj[x][i] == d)
      delEdge(x,i), invert(i,c,d), ae(x,i,c); }
  void ae(int u, int v) {
    // check if you can add edge w/o doing any work
    assert(N); ckmax(maxDeg,max(++deg[u],++deg[v]));
    auto a = genCol(u), b = genCol(v);
    FOR(i,1,maxDeg+2) if (!a[i] && !b[i])
      return ae(u,v,i);
    V<bool> use(N); vi fan = {v}; use[v] = 1;
    while (1) {
      auto col = genCol(fan.bk);
      if (sz(fan) > 1) col[adj[fan.bk][u]] = 0;
      int i=0; while (i<N && (use[i] || col[adj[u][i]])) i++;
      if (i < N) fan.pb(i), use[i] = 1;
      else break;
    }
    int c = freeCol(u), d = freeCol(fan.bk); invert(u,d,c);
    int i = 0; while (i < sz(fan) && genCol(fan[i])[d]
      && adj[u][fan[i]] != d) i ++;
    assert (i != sz(fan));
    F0R(j,i) ae(u,fan[j],delEdge(u,fan[j+1]));
    ae(u,fan[i],d);
  }
};
```

DirectedMST.h

Description: Chu-Liu-Edmonds algorithm. Computes minimum weight directed spanning tree rooted at r, edge from $par[i] \to i$ for all $i \neq r$. Use DSU with rollback if need to return edges.
Time: $\mathcal{O}(M \log M)$

"DSUrb.h" 5d5c10, 61 lines

```
struct Edge { int a, b; ll w; };
struct Node { // lazy skew heap node
  Edge key; Node *l, *r; ll delta;
  void prop() {
    key.w += delta;
    if (l) l->delta += delta;
    if (r) r->delta += delta;
    delta = 0;
  }
  Edge top() { prop(); return key; }
};
Node *merge(Node *a, Node *b) {
  if (!a || !b) return a ?: b;
  a->prop(), b->prop();
  if (a->key.w > b->key.w) swap(a, b);
  swap(a->l, a->r = merge(b, a->r));
  return a;
}
void pop(Node*& a) { a->prop(); a = merge(a->l, a->r); }

pair<ll,vi> dmst(int n, int r, const vector<Edge>& g) {
  DSUrb dsu; dsu.init(n);
  vector<Node*> heap(n); // store edges entering each vertex
  // in increasing order of weight
  each(e,g) heap[e.b] = merge(heap[e.b], new Node{e});
  ll res = 0; vi seen(n,-1); seen[r] = r;
  vpi in(n,{-1,-1}); // edge entering each vertex in MST
  vector<pair<int,vector<Edge>>> cycs;
  F0R(s,n) {
    int u = s, w;
    vector<pair<int,Edge>> path;
    while (seen[u] < 0) {
      if (!heap[u]) return {-1,{}};
      seen[u] = s;
      Edge e = heap[u]->top(); path.pb({u,e});
      heap[u]->delta -= e.w, pop(heap[u]);
      res += e.w, u = dsu.get(e.a);
      if (seen[u] == s) { // found cycle, contract
        Node* cyc = 0; cycs.eb();
        do {
          cyc = merge(cyc, heap[w = path.bk.f]);
          cycs.bk.s.pb(path.bk.s);
          path.pop_back();
        } while (dsu.unite(u,w));
        u = dsu.get(u); heap[u] = cyc, seen[u] = -1;
        cycs.bk.f = u;
      }
    }
    each(t,path) in[dsu.get(t.s.b)] = {t.s.a,t.s.b};
  } // found path from root to s, done
  while (sz(cycs)) { // expand cycs to restore sol
    auto c = cycs.bk; cycs.pop_back();
    pi inEdge = in[c.f];
    each(t,c.s) dsu.rollback();
    each(t,c.s) in[dsu.get(t.b)] = {t.a,t.b};
    in[dsu.get(inEdge.s)] = inEdge;
  }
  vi par(n); F0R(i,n) par[i] = in[i].f;
  // i == r ? in[i].s == -1 : in[i].s == i
  return {res,par};
}
```

LCT.h

Description: Link-Cut Tree. Given a function $f(1 \ldots N) \to 1 \ldots N$, evaluates $f^b(a)$ for any a, b. `sz` is for path queries; `sub`, `vsub` are for subtree queries. `x->access()` brings `x` to the top and propagates it; its left subtree will be the path from `x` to the root and its right subtree will be empty. Then `sub` will be the number of nodes in the connected component of `x` and `vsub` will be the number of nodes under `x`. Use `makeRoot` for arbitrary path queries.
Usage: `FOR(i,1,N+1)LCT[i]=new snode(i); link(LCT[1],LCT[2],1);`
Time: $\mathcal{O}(\log N)$

e24bf7, 115 lines

```
typedef struct snode* sn;
struct snode { //////// VARIABLES
  sn p, c[2]; // parent, children
  sn extra; // extra cycle node for "The Applicant"
  bool flip = 0; // subtree flipped or not
  int val, sz; // value in node, # nodes in current splay tree
  int sub, vsub = 0; // vsub stores sum of virtual children
  snode(int _val) : val(_val) {
    p = c[0] = c[1] = extra = NULL; calc(); }
  friend int getSz(sn x) { return x?x->sz:0; }
  friend int getSub(sn x) { return x?x->sub:0; }
  void prop() { // lazy prop
    if (!flip) return;
    swap(c[0],c[1]); flip = 0;
    F0R(i,2) if (c[i]) c[i]->flip ^= 1;
  }
  void calc() { // recalc vals
    F0R(i,2) if (c[i]) c[i]->prop();
    sz = 1+getSz(c[0])+getSz(c[1]);
    sub = 1+getSub(c[0])+getSub(c[1])+vsub;
  }
  //////// SPLAY TREE OPERATIONS
  int dir() {
    if (!p) return -2;
    F0R(i,2) if (p->c[i] == this) return i;
    return -1; // p is path-parent pointer
  } // -> not in current splay tree
  // test if root of current splay tree
  bool isRoot() { return dir() < 0; }
  friend void setLink(sn x, sn y, int d) {
    if (y) y->p = x;
    if (d >= 0) x->c[d] = y; }
  void rot() { // assume p and p->p propagated
    assert(!isRoot()); int x = dir(); sn pa = p;
    setLink(pa->p, this, pa->dir());
```

```
    setLink(pa, c[x^1], x); setLink(this, pa, x^1);
    pa->calc();
  }
  void splay() {
    while (!isRoot() && !p->isRoot()) {
      p->p->prop(), p->prop(), prop();
      dir() == p->dir() ? p->rot() : rot();
      rot();
    }
    if (!isRoot()) p->prop(), prop(), rot();
    prop(); calc();
  }
  sn fbo(int b) { // find by order
    prop(); int z = getSz(c[0]); // of splay tree
    if (b == z) { splay(); return this; }
    return b < z ? c[0]->fbo(b) : c[1] -> fbo(b-z-1);
  }
  //////// BASE OPERATIONS
  void access() { // bring this to top of tree, propagate
    for (sn v = this, pre = NULL; v; v = v->p) {
      v->splay(); // now switch virtual children
      if (pre) v->vsub -= pre->sub;
      if (v->c[1]) v->vsub += v->c[1]->sub;
      v->c[1] = pre; v->calc(); pre = v;
    }
    splay(); assert(!c[1]); // right subtree is empty
  }
  void makeRoot() {
    access(); flip ^= 1; access(); assert(!c[0] && !c[1]); }
  //////// QUERIES
  friend sn lca(sn x, sn y) {
    if (x == y) return x;
    x->access(), y->access(); if (!x->p) return NULL;
    x->splay(); return x->p?:x; // y was below x in latter case
  } // access at y did not affect x -> not connected
  friend bool connected(sn x, sn y) { return lca(x,y); }
  // # nodes above
  int distRoot() { access(); return getSz(c[0]); }
  sn getRoot() { // get root of LCT component
    access(); sn a = this;
    while (a->c[0]) a = a->c[0], a->prop();
    a->access(); return a;
  }
  sn getPar(int b) { // get b-th parent on path to root
    access(); b = getSz(c[0])-b; assert(b >= 0);
    return fbo(b);
  } // can also get min, max on path to root, etc
  //////// MODIFICATIONS
  void set(int v) { access(); val = v; calc(); }
  friend void link(sn x, sn y, bool force = 0) {
    assert(!connected(x,y));
    if (force) y->makeRoot(); // make x par of y
    else { y->access(); assert(!y->c[0]); }
    x->access(); setLink(y,x,0); y->calc();
  }
  friend void cut(sn y) { // cut y from its parent
    y->access(); assert(y->c[0]);
    y->c[0]->p = NULL; y->c[0] = NULL; y->calc(); }
  friend void cut(sn x, sn y) { // if x, y adj in tree
    x->makeRoot(); y->access();
    assert(y->c[0] == x && !x->c[0] && !x->c[1]); cut(y); }
};
sn LCT[MX];

//////// THE APPLICANT SOLUTION
void setNex(sn a, sn b) { // set f[a] = b
  if (connected(a,b)) a->extra = b;
  else link(b,a); }
void delNex(sn a) { // set f[a] = NULL
  auto t = a->getRoot();
  if (t == a) { t->extra = NULL; return; }
  cut(a); assert(t->extra);
  if (!connected(t,t->extra))
    link(t->extra,t), t->extra = NULL;
}
sn getPar(sn a, int b) { // get f^b[a]
  int d = a->distRoot(); if (b <= d) return a->getPar(b);
  b -= d+1; auto r = a->getRoot()->extra; assert(r);
  d = r->distRoot()+1; return r->getPar(b%d);
}
```